

CS-499: 3-2 Enhancement One Narrative Software Design and Engineering

Syed Zeeshan Hyder

Southern New Hampshire University

CS-499: Computer Science Capstone

Professor Joseph Martinez

July 11th, 2026

Briefly describe the artifact. What is it? When was it created?

The artifact I selected for this assignment and the next two is the Travlr Getaways Full-Stack Web Application, originally created during CS 465: Full Stack Development 1, a class I took in my previous term, May - June. The project, put simply, is a full stack travel booking application which was built using the MEAN stack, consisting of an Angular administrator application, a Node.js and Express backend, and finally a MongoDB database which was done through Mongoose. The application also uses JWT authentication, which allows managing administrator access and follows the MVC architecture that separates routes and controllers.

At the time of submitting the original application, it did meet the requirements presented in CS 465, but looking at it through the fresh lens being this capstone allowed me to evaluate it from a professional software engineering perspective. Instead of building a bunch of new functionality, this enhancement is more focused on improving the maintainability, security, and architecture of the application, while also attempting to better user experience.

Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in software development? How was the artifact improved?

I believe that this artifact, in specific, is a representation of my strongest software engineering abilities throughout my computer science program thus far, and unlike many earlier projects which focused on one concept at a time, Travlr is a combination of frontend, backend, authentication, database integration, and architecture all into a single project. It gave me an opportunity to demonstrate my ability of improving and maintaining an existing application instead of always creating a new one, which is something I haven't been able to try or showcase yet.

This project demonstrates my ability to design RESTful APIs, develop Angular components, organize backend controllers, integrate MongoDB using Mongoose, and implement secure authentication using JSON Web Tokens.

The largest improvement was strengthening the application's security, as admin API endpoints used to rely only on the frontend being able to hide admin functionality, which meant if a normal user were to somehow send a request, it would still be sent directly to the backend. My proposed fix for this was implementing JWT authentication middleware to protect the administrative routes, then configuring Angular to automatically attach authentication tokens using an HTTP interceptor, which made it so only authenticated users (me) were able to perform admin actions.

The application's maintainability was also improved by reorganizing portions of the backend controller logic into reusable validation functions; duplicate trip codes are now detected prior to those records being created, and more useful and meaningful HTTP status codes and error messages are now returned when needed. While on the frontend, the authentication was improved by removing the timeout-based login approach, which now gives instant login/logout updates, improves validation feedback, and shows clearer messages to users if an error were to occur.

Did you meet the course outcomes you planned to meet with this enhancement in Module One? Do you have any updates to your outcome-coverage plans?

I'd like to say yes, this enhancement has met the outcomes related to software engineering, communication, and secure development that I planned to address in Module One. The biggest outcome shown through this enhancement is applying professional software engineering techniques to improve an existing application instead of simply adding new features. I recognized parts of the backend, added reusable logic, made better API responses, added JWT authentication, and

strengthened the connection between the frontend and the backend, which made the application more secure, easier to maintain, and a better show of how software is developed in a professional environment.

I also gained a better insight of security during this enhancement, where originally the administrative actions relied on the frontend to keep hidden, improving the application by protecting backend routes with JWT authentication, making the administrative actions actually secure, instead of pretending to be. Changes like such reduced the risk the original application came with and made sure I treated backend validation and authorization as part of the overall application.

The code review as well as this narrative also gave me the opportunity to communicate my technical decisions in a clear way, as being able to explain why changes were made is just as, if not more important than, making the changes themselves, as I should be able to communicate design decisions with coworkers, teammates, managers, and potential stakeholders.

As of now, I do not have any major changes to my original enhancement plan. The next enhancement will focus more on improving the applications algorithms by adding features such as searching, sorting, and pagination, and the final enhancement will focus more on the database design and overall performance, while touching up any other minor issues I see with the application.

Reflect on the process of enhancing and modifying the artifact. What did you learn as you were creating it and improving it? What challenges did you face?

I would say while working on this enhancement, I have gained a new perspective than the one I had when I originally built the application a month ago in CS 465. Back then my goal was simply to create a working full stack application that met the requirements, and with the perfect grades I got with every submission, I assumed it was the best it could possibly get, then during the capstone, I noticed that there was a lot that I could change, so instead of building from scratch I approached the project as someone who had to improve it without breaking any of the current functionality.

I learnt how important it was to write code that was easy to maintain, not to say it wasn't easy to maintain prior, but while I reorganized controller logic and improved authentication flow, I noticed how small changes to the structure of the code would make the application easier to understand and expand on later if I was to do so. So rather than only thinking about whether the code worked, I started to think about how easy it would be for someone else, or even myself a few months from now not being so comfortable with the code, to come back to and continue working on.

I also feel as though I better understand the relationship between the backend and frontend. The Angular application can provide immediate feedback to improve the user experience, but the backend still has to validate incoming requests from the user, as they could bypass the frontend entirely, so implementing JWT authentication on both sides of the application showed why security should be built into the backend instead of relying only on the user interface.

Authentication touched multiple parts of the project, and a small change in one place could easily break something somewhere else, so a challenge for sure was improving the authentication system without affecting the rest of the application. I had to push the tedious yet needed mindset of testing the application after every significant change, instead of waiting until I finished, and was tasked with tracing my footsteps to see where I messed up.

I walked into this enhancement thinking I'd add a few features and call it a day, but it may have been more challenging than building something from scratch, as it required planning, constant testing, and an understanding of how different parts of my application worked together.